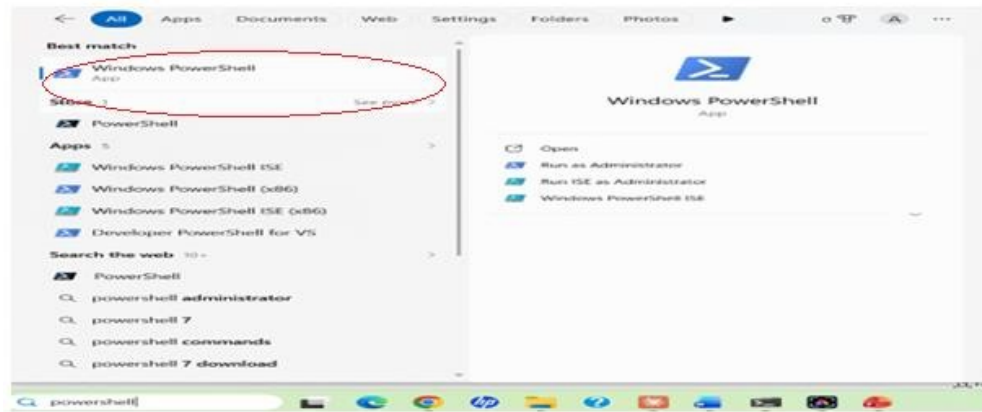


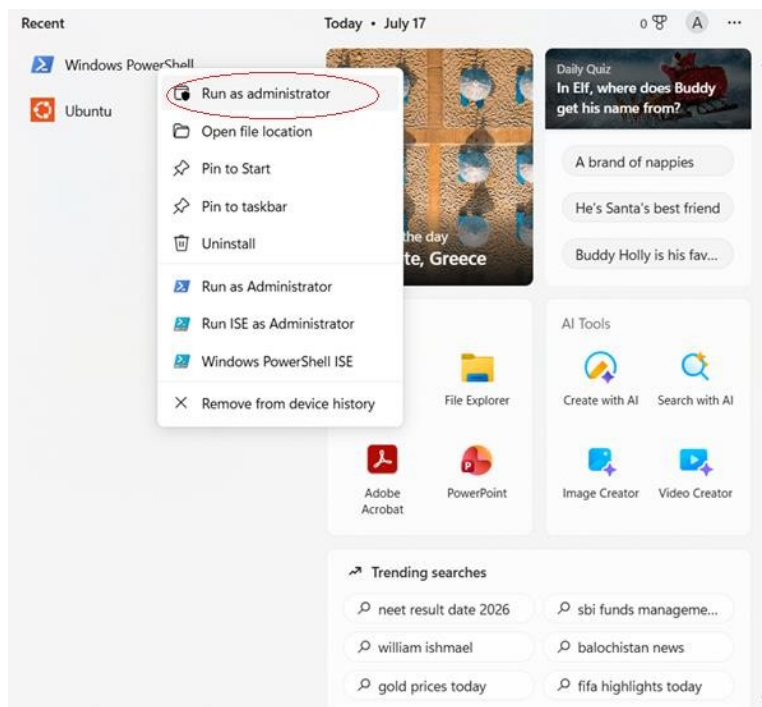
# OSSP – SKILL 1

*Installing Linux (WSL + Ubuntu), configuring the GCC compiler, compiling a first C program, and exploring processes with `fork()`, `exec()`, and system calls.*

Searched for Windows PowerShell in the Start menu.



Right-clicked Windows PowerShell and selected Run as administrator.



Ran `wsl --install` in PowerShell to install WSL along with Ubuntu, then restarted the PC when prompted.

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\samhi> wsl --install
```

After the restart, Ubuntu opened and asked me to create a UNIX username and password:

```
Enter new UNIX username: manu
New password: *****
Retype new password: *****
```

Updated the package lists with `sudo apt update` so everything is current.

```
manu@Manasvi: ~$ sudo apt update
[sudo: authenticate] Password:
Get:1 http://security.ubuntu.com/ubuntu resolute-security InRelease [137 kB]
Hit:2 http://archive.ubuntu.com/ubuntu resolute InRelease
Get:3 http://archive.ubuntu.com/ubuntu resolute-updates InRelease [137 kB]
Get:4 http://security.ubuntu.com/ubuntu resolute-security/main amd64 Packages [424 kB]
Get:5 http://archive.ubuntu.com/ubuntu resolute-backports InRelease [137 kB]
Get:6 http://archive.ubuntu.com/ubuntu resolute-updates/main amd64 Packages [549 kB]
Get:7 http://security.ubuntu.com/ubuntu resolute-security/main Translation-en [97.9 kB]
Get:8 http://security.ubuntu.com/ubuntu resolute-security/main amd64 Components [46.7 kB]
Get:9 http://security.ubuntu.com/ubuntu resolute-security/universe amd64 Packages [159 kB]
Get:10 http://security.ubuntu.com/ubuntu resolute-security/universe amd64 Components [43.1 kB]
Get:11 http://security.ubuntu.com/ubuntu resolute-security/restricted amd64 Packages [361 kB]
Get:12 http://security.ubuntu.com/ubuntu resolute-security/restricted Translation-en [70.0 kB]
Get:13 http://archive.ubuntu.com/ubuntu resolute-updates/main Translation-en [128 kB]
Get:14 http://archive.ubuntu.com/ubuntu resolute-updates/main amd64 Components [98.0 kB]
Get:15 http://archive.ubuntu.com/ubuntu resolute-updates/universe amd64 Packages [252 kB]
Get:16 http://archive.ubuntu.com/ubuntu resolute-updates/universe Translation-en [84.4 kB]
Get:17 http://archive.ubuntu.com/ubuntu resolute-updates/universe amd64 Components [187 kB]
Get:18 http://archive.ubuntu.com/ubuntu resolute-backports/universe amd64 Packages [3132 B]
Get:19 http://archive.ubuntu.com/ubuntu resolute-backports/universe Translation-en [7988 B]
Get:20 http://archive.ubuntu.com/ubuntu resolute-backports/universe amd64 Components [1056 B]
Fetched 2923 kB in 3s (959 kB/s)
Reading package lists... 7%
```

Installed the GCC C compiler using `sudo apt install build-essential` (it was already the newest version).

```
manu@Manasvi:~$ sudo apt install build-essential
build-essential is already the newest version (12.12ubuntu2.26.04.2).
Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 40
manu@Manasvi:~$
```

Checked the compiler with `gcc --version` — GCC 15.2.0 is installed and working.

```
manu@Manasvi:~$ gcc --version
gcc (Ubuntu 15.2.0-16ubuntu1) 15.2.0
Copyright (C) 2025 Free Software Foundation, Inc.
This is free software; see the source for copying conditions. There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

manu@Manasvi:~$ |
```

Created the project folder `ossp_shell` with `src`, `include`, and `obj` subfolders, then listed them with `ls`.

```
manu@Manasvi:~$ cd os_lab
manu@Manasvi:~/os_lab$ mkdir ossp_shell
manu@Manasvi:~/os_lab$ cd ossp_shell
manu@Manasvi:~/os_lab/ossp_shell$ mkdir src
manu@Manasvi:~/os_lab/ossp_shell$ mkdir include
manu@Manasvi:~/os_lab/ossp_shell$ mkdir obj
manu@Manasvi:~/os_lab/ossp_shell$ ls
include obj src
manu@Manasvi:~/os_lab/ossp_shell$ |
```

Wrote a simple C program in `src/hello.c` using `nano` that prints a message to the screen.

```
GNU nano 8.7.1 src/hello.c
#include <stdio.h>

int main() {
    printf("Hello,OSSP!GCC is configured\n");
    return 0;
}
```

Compiled the program directly with `gcc src/hello.c` and ran it with `./a.out`, which printed the output.

```
manu@Manasvi:~/os_lab/ossp_shell$ gcc src/hello.c
manu@Manasvi:~/os_lab/ossp_shell$ ./a.out
Hello,OSSP!GCC is configured
manu@Manasvi:~/os_lab/ossp_shell$ |
```

Created a Makefile with a simple rule that builds `hello` from `src/hello.c`.

```
GNU nano 8.7.1 Makefile *
hello:
    gcc src/hello.c -o hello|
```

Built the program with `make` (it reported `hello` was already up to date) and ran `./hello`, giving the same output.

```

manu@Manasvi:~/os_lab/ossdp_shell$ nano Makefile
manu@Manasvi:~/os_lab/ossdp_shell$ make
make: 'hello' is up to date.
manu@Manasvi:~/os_lab/ossdp_shell$ ./hello
Hello, OSSP!GCC is configured
manu@Manasvi:~/os_lab/ossdp_shell$ |

```

## Understanding shell architecture

A shell runs in a simple loop: it reads a command, parses it, then creates a child process with `fork()` and runs the command with `exec()`, while the parent process waits (`wait()`) for the child to finish before showing the prompt again.

## Exploring processes: `fork()`, `exec()`, and system calls

Wrote a program (`forkexec.c`) that uses `fork()` to create a child process; the child then uses `exec()` to run the `ls` command, while the parent waits for it to finish.

```

GNU nano 8.7.1                                     forkexec.c

#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    int pid = fork();                                // create a child process

    if (pid == 0) {                                   // CHILD
        printf("Child (PID %d) running 'ls' using exec:\n", getpid());
        execlp("ls", "ls", NULL);                   // replace the child with the 'ls' command
    } else {                                          // PARENT
        wait(NULL);                                  // wait for the child to finish
        printf("Parent (PID %d): child has finished.\n", getpid());
    }
    return 0;
}

```

Compiled it with `gcc forkexec.c` and ran `./a.out` — the child (PID 1622) ran `ls` through `exec`, and the parent (PID 1621) waited until the child finished.

```

manu@Manasvi:~/os_lab$ gcc forkexec.c
manu@Manasvi:~/os_lab$ ./a.out
Child (PID 1622) running 'ls' using exec:
Practical-2.c  Practical_1.c  forkexec.c  ossdp_shell
Practical-3.c  a.out          input.txt  output.txt
Parent (PID 1621): child has finished.
manu@Manasvi:~/os_lab$ |

```

Used `ps` to inspect the process tree — it shows how every process descends from `systemd`, the first process started at boot.

```
manu@Manasvi:~/os_lab$ pstree
systemd--agetty
        |
        |--chronyd-starter--chronyd--chronydc
        |
        |--cron
        |
        |--dbus-daemon
        |
        --init-systemd(Ubuntu)---SessionLeader---Relay(312)--bash--+
++                                |
                                |--init---{init}
                                |--login---bash
                                --{init-systemd(Ubuntu)}
                                |
                                |--networkd-dispatcher
                                |--polkitd---3*[{polkitd}]
                                |--rsyslogd---3*[{rsyslogd}]
                                |--systemd---(sd-pam)
                                |--systemd-journald
                                |--systemd-logind
                                |--systemd-resolved
                                |--systemd-udevd
                                |--unattended-upgr---{unattended-upgr}
                                --wsl-pro-service---7*[{wsl-pro-service}]
manu@Manasvi:~/os_lab$
```

Used strace ./a.out to trace the system calls the program makes. The first part shows the program start-up calls (execve, mmap, openat, and so on).

```
manu@Manasvi:/~/os_lab$ strace ./a.out
execve("./a.out", ["/a.out"], 0x7ffdf7b4bbf40 /* 26 vars */) = 0
brk(NULL)                               = 0x5c010f737000
mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7f61482d4000
access("/etc/ld.so.preload", R_OK)      = -1 ENOENT (No such file or directory)
openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
fstat(3, {st_mode=S_IFREG|0644, st_size=27891, ...}) = 0
mmap(NULL, 27891, PROT_READ, MAP_PRIVATE, 3, 0) = 0x7f61482cd000
close(3)                                = 0
openat(AT_FDCWD, "/usr/lib/x86_64-linux-gnu/libc.so.6", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\3\0\0\0\0\0\0\0\0\0\0\0\0>\0\1\0\0\0 \250\2\0\0\0\0\0...", 832) = 832
pread64(3, "\6\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0...", 840, 64) = 840
fstat(3, {st_mode=S_IFREG|0755, st_size=2186512, ...}) = 0
pread64(3, "\6\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0...", 840, 64) = 840
mmap(NULL, 2231696, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) = 0x7f6148000000
mmap(0x7f6148028000, 1671168, PROT_READ|PROT_EXEC, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x28000) = 0x7f6148028000
mmap(0x7f61481c0000, 319488, PROT_READ, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x1c0000) = 0x7f61481c0000
mmap(0x7f614820e000, 24576, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x20d000) = 0x7f614820e000
mmap(0x7f6148214000, 52624, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_ANONYMOUS, -1, 0) = 0x7f6148214000
close(3)                                = 0
mmap(NULL, 12288, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7f61482ca000
arch_prctl(ARCH_SET_FS, 0x7f61482ca740) = 0
set_tid_address(&0x7f61482cad68)        = 1709
set_robust_list(&0x7f61482caa20, 24)    = 0
rseq(0x7f61482ca6a0, 0x20, 0, 0x53053053) = 0
mprotect(0x7f614820e000, 16384, PROT_READ) = 0
mprotect(0x5c00dd6f000, 4096, PROT_READ) = 0
mprotect(0x7f6148319000, 8192, PROT_READ) = 0
prlimit64(0, RLIMIT_STACK, NULL, {rlim_cur=8192*1024, rlim_max=RLIM64_INFINITY}) = 0
getrandom("\x9e\x1f\x2d\xb6\xb0\xe8\xc6\x66". 8, GRND_NONBLOCK) = 8
```

The second part shows the key process calls: clone (from fork), wait4 (the parent waiting), write (the printf output), and exit\_group.

```

getrand48( (x1*(x2*(x3*(x4*(x5*(x6*(x7*(x8 / 8, rand_tseed23) - 1
munmap(0x7f61482cd000, 27891) = 0
rt_sigprocmask(SIG_BLOCK, ~[], [], 8) = 0
clone(child_stack=NULL, flags=CLONE_CHILD_CLEARTID|CLONE_CHILD_SETTID|SIGCHLD, child_tidptr=0x7f61482caa10) = 1710
rt_sigprocmask(SIG_SETMASK, [], Child (PID 1710) running 'ls' using exec:
, NULL, 8) = 0
wait4(-1 Practical-2.c Practical_1.c forkexec.c ossp_shell
Practical-3.c a.out input.txt output.txt
, NULL, 0, NULL) = 1710
--- SIGCHLD {si_signo=SIGCHLD, si_code=CLD_EXITED, si_pid=1710, si_uid=1000, si_status=0, si_utime=0, si_stime=0} ---
getpid() = 1709
fstat(1, {st_mode=S_IFCHR|0620, st_rdev=makedev(0x88, 0), ...}) = 0
brk(NULL) = 0x5c010f737000
brk(0x5c010f758000) = 0x5c010f758000
write(1, "Parent (PID 1709): child has fin"..., 39Parent (PID 1709): child has finished.
) = 39
exit_group(0) = ?
+++ exited with 0 +++
manu@Manasvi:~/os_lab$ |

```